PYTHON PROJECT FILE

Sentiment *Analyser*

Teaching a computer to read the feelings hidden inside words.



POSITIVE



NEUTRAL



NEGATIVE

pandas | TextBlob | WordCloud | matplotlib

Natural Language Processing • Python • Data Visualisation

Contents

01 Introduction 3

02 Objectives of the Project 3

03 Tools and Requirements 4

04 Important Terms Explained Simply 5

05 How the Program Works 6

06 Source Code 7

07 Code Explanation 8

08 Output 9

09 Understanding the Results 11

10 Advantages, Limitations and Future Scope 11

11 Conclusion 12

01 Introduction

Every day people write millions of reviews, comments and feedback about products and services. Reading all of them one by one is impossible. But the **feelings** hidden inside those words are very valuable to a company: are customers **happy**, **unhappy** or just **okay**?

Sentiment Analysis is the method of teaching a computer to find the feeling (positive, negative or neutral) in a piece of text. This project reads a file full of customer sentences and automatically:

- labels every sentence as **Positive**, **Negative** or **Neutral**,
- saves the result in an **Excel** file,
- picks out only the **negative** sentences, and
- draws a **Word Cloud** showing the most used words in the negative feedback.

IN ONE LINE

A small tool that reads customer feedback and tells you what people are unhappy about.

02 Objectives of the Project

Learn Python

Practise functions, conditions and pandas on a real data file.

Understand NLP

A first taste of Natural Language Processing: making computers understand human language.

Classify Text

Label each sentence as Positive, Negative or Neutral using TextBlob.

Filter Data

Use pandas to pick only the rows we need (negative reviews).

Visualise Words

Create a Word Cloud to see the most common words at a glance.

Save the Results

Export the final table to an Excel file anyone can open.

03 Tools and Requirements

Python Libraries Used

Library	Imported as	What it does in this project
pandas	pd	Reads the CSV file, stores data in a table (DataFrame), filters rows and saves to Excel.
matplotlib	plt	Shows the Word Cloud picture with a title.
seaborn	sns	Makes graphs prettier. (Imported for future use.)
wordcloud	WordCloud	Creates a picture where common words look bigger.
textblob	TextBlob	Understands a sentence and gives it a score (polarity).
openpyxl	-	Lets pandas write an .xlsx Excel file.

Dataset and Software

Item	Details
Dataset file	Sentiment_Analysis .csv (one column named **Sentences**, each row is one customer opinion)
Language	Python 3.x
Editor	Jupyter Notebook (Anaconda)
Output files	SentimentAnalysisbyYatin.xlsx and the Word Cloud picture
Hardware	Any normal computer with at least 4 GB RAM

INSTALL Open Command Prompt and type: `pip install pandas matplotlib seaborn wordcloud textblob openpyxl`

04 Important Terms Explained Simply

Sentiment Analysis
Finding the feeling behind a text: is the writer happy (positive), unhappy (negative) or neutral?

Polarity Score
A number from **-1** to **+1** given by TextBlob. Above 0 = positive, below 0 = negative, exactly 0 = neutral.

TextBlob
A Python library that already knows the feeling of thousands of English words. It reads a sentence and gives its polarity.

DataFrame
A table in pandas with rows and columns, just like an Excel sheet.

Word Cloud
A picture made of words. The more often a word appears, the **bigger** it looks.

CSV File
A simple file that stores table data as plain text, with values separated by commas.

EXAMPLE

"The service was outstanding" gets a polarity above 0, so it is **Positive**.
"The product was dreadful" gets a polarity below 0, so it is **Negative**.
A sentence with no feeling words gets exactly 0, so it is **Neutral**.

05 How the Program Works

The program runs from top to bottom, one step after another. Follow the journey of the data:

1. **Import the libraries**
   Bring in pandas, matplotlib, WordCloud and TextBlob
2. **Load the dataset**
   Read Sentiment_Analysis .csv into a DataFrame called data
3. **Define get_sentiment()**
   TextBlob gives a polarity score and we turn it into a label
4. **Apply it to every sentence**
   A new column called Sentiment is added to the table
5. **Save to Excel**
   Export the table as SentimentAnalysisbyYatin.xlsx
6. **Filter negative sentences**
   Keep only the rows labelled Negative
7. **Join into one paragraph**
   Combine all negative sentences into one big text
8. **Create the Word Cloud**
   Count the words so that common words look bigger
9. **Show the picture**
   Display it with matplotlib. Done!

06 Source Code

The program is written in nine small Jupyter cells. Each cell does one job.

CELL 1 • IMPORTING LIBRARIES

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
from textblob import TextBlob # to understand meaning of the sentences
```

CELL 2 • LOADING THE DATASET

```
#Load the dataset
data=pd.read_csv("Sentiment_Analysis .csv")
data.head(10)
```

CELL 3 • THE SENTIMENT FUNCTION

```
# define the sentiment analysis function
def get_sentiment(text):
    meaning=TextBlob(text)

    if meaning.sentiment.polarity>0:
        return "Positive"
    elif meaning.sentiment.polarity<0:
        return "Negative"
    else:
        return "Neutral"
```

CELL 4 • APPLYING THE FUNCTION

```
data["Sentiment"]=data["Sentences"].apply(get_sentiment)
```

CELL 5 • SAVING TO EXCEL

```
%pip install openpyxl
data.to_excel("SentimentAnalysisbyYatin.xlsx",index=False)
```

CELL 6 • CHECKING THE RESULT

```
data.head()
```

CELL 7 • FILTERING NEGATIVE SENTENCES

```
#filter only negative views
negative_reviews=data[data["Sentiment"]=="Negative"]
negative_reviews.head(10)
```

CELL 8 • JOINING INTO A PARAGRAPH

```
#combine all the negative sentences into one string/paragraph
negative_para=" ".join(negative_reviews["Sentences"])
print(negative_para)
```

CELL 9 • CREATING THE WORD CLOUD

```
# create a word cloud for negative paragraph
wordcloud=WordCloud(width=500,height=300,background_color="black").generate(negative_para)
plt.figure(figsize=(10,4))
plt.imshow(wordcloud,interpolation="bilinear")
plt.title("Most Common Words in NEGATIVE sentences")
plt.axis("off")
plt.show()
```

07 Code Explanation

CELL 1

Importing libraries

import brings ready-made tools into our program. pandas handles data, matplotlib shows pictures, WordCloud makes the word picture and TextBlob understands sentences.

CELL 2

Loading the dataset

pd.read_csv() reads the file into a DataFrame called **data**. **data.head(10)** shows the first 10 rows so we can check the file loaded correctly.

CELL 3

get_sentiment() function

Gives one sentence to **TextBlob** and checks **sentiment.polarity**. More than 0 returns "Positive", less than 0 returns "Negative", otherwise "Neutral".

CELL 4

Applying the function

.apply(get_sentiment) runs the function on every sentence in the **Sentences** column and stores the answers in a new column **Sentiment**.

CELL 5

Saving to Excel

%pip install openpyxl installs the helper library. **to_excel()** saves the table as an Excel file. **index=False** stops pandas adding an extra column of row numbers.

CELL 6

Checking the result

data.head() shows the first 5 rows with the new Sentiment column.

CELL 7

Filtering negative sentences

data[data["Sentiment"]=="Negative"] keeps only rows where the sentiment is Negative and stores them in **negative_reviews**.

CELL 8

Joining into one paragraph

" ".join(...) joins all negative sentences into one long text with a space between them. The WordCloud needs one single text to count words.

CELL 9

Creating the Word Cloud

WordCloud(width=500, height=300, background_color="black") sets size and colour. **.generate()** counts the words and builds the picture. **plt.imshow()** shows it, **plt.axis("off")** hides the axis lines and **plt.show()** displays the final picture.

08 Output

Step 1: Sentences with their sentiment (first 5 rows)

#	Sentences	Sentiment
0	The product's exceptional quality and performa...	Positive
1	Poor customer support and the product's subpar...	Negative
2	The service met standard requirements without ...	Positive
3	Outstanding customer service accompanied the p...	Positive
4	The product's failure to meet basic requiremen...	Negative

Long sentences are cut short with "..." in the table display.

Step 2: Only the negative sentences

#	Sentences	Sentiment
1	Poor customer support and the product's subpar...	Negative
4	The product's failure to meet basic requiremen...	Negative
7	Poor customer service and a faulty product led...	Negative
8	The service provided was average, meeting mini...	Negative
13	Dreadful customer support coupled with a defec...	Negative
16	Severe product shortcomings led to considerabl...	Negative
17	The service was standard, meeting basic needs ...	Negative
19	Disappointing product quality and poor custome...	Negative
20	The service provided was average, meeting basi...	Negative

Step 3: All negative sentences joined into one paragraph (starting part)

CONSOLE OUTPUT

```
Poor customer support and the product's subpar performance led to frustration and
disappointment, regretting the purchase entirely. The product's failure to meet basic
requirements resulted in immense inconvenience and frustration, causing regret over the
purchase. Poor customer service and a faulty product led to significant inconvenience and
frustration, resulting in regret over the purchase decision. ...
```

EXCEL All sentences with their Sentiment are also saved in **SentimentAnalysisbyYatin.xlsx**.

Step 4: The Word Cloud

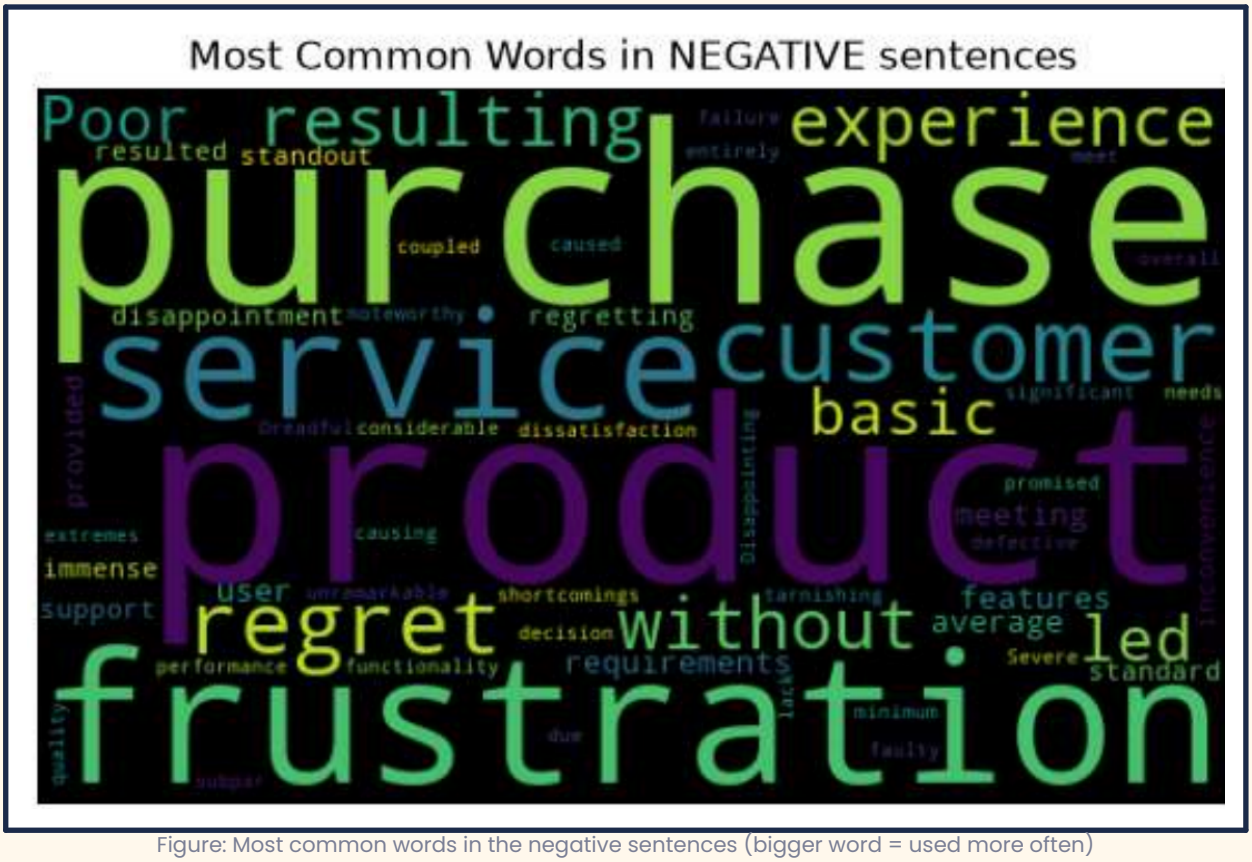


Figure: Most common words in the negative sentences (bigger word = used more often)

09 Understanding the Results

Reading the Word Cloud

- The biggest words are **purchase**, **product**, **frustration**, **service**, **customer** and **regret**. These appear most often in the negative sentences.
- **Product** and **service / customer** show that people complain mainly about product quality and customer support.
- **Frustration**, **regret** and **disappointment** show the emotions of unhappy customers.
- Smaller words like **faulty**, **defective**, **subpar**, **shortcomings** and **inconvenience** point to the exact problems.
- A company can use this to decide what to fix first: better product quality and better customer service.

NOTICE

Some sentences like *"The service provided was average, meeting minimum requirements..."* (rows 8, 17 and 20) sound neutral but were still labelled **Negative**. TextBlob judges by individual words and does not fully understand the whole sentence. No tool is 100% perfect.

10 Advantages, Limitations and Future Scope

Advantages

- Very short and simple code
- Works on hundreds of sentences in seconds
- Result is saved to Excel
- Word Cloud makes problem areas easy to see
- No need to train any model

Limitations

- TextBlob works only for English
- Can misjudge average or mixed sentences
- Cannot understand sarcasm
- Common words ("without", "led") also appear in the cloud
- File name must be typed exactly

Future Scope (ideas to improve)

- Count Positive, Negative and Neutral sentences and show a **bar or pie chart** (seaborn is already imported).
- Make Word Clouds for **positive** sentences too and compare them.
- Remove common "stop words" to make the Word Cloud more meaningful.
- Add a **Neutral** range so that very small scores like 0.05 count as Neutral.
- Use advanced models like VADER or BERT for better accuracy.
- Make a small app where the user types any sentence and gets its sentiment instantly.

11 Conclusion

The **Sentiment Analyser** project shows how Python can understand human feelings hidden in text. With only a few lines of code, we loaded a dataset, labelled every sentence as Positive, Negative or Neutral, saved the results to Excel and created a Word Cloud of the negative feedback.

While making this project I learned about **pandas**, **functions**, **TextBlob**, **filtering data** and **word clouds**. I also learned that tools can make small mistakes, so results should always be checked by a human.

This project is a first step into **Natural Language Processing**, and any company can use it to understand what its customers really think.

Thank You!